

CO2_AT2-SCENARIO BASED

QUESTIONS

Q1

Smart Hospital Patient Registration System

20 marks

Q2

Online Library Book Management System

20 marks

2/2 answered

Q1 Smart Hospital Patient Registration System

20 marks

Smart Hospital Patient Registration System

A multi-specialty hospital wants to digitize its outpatient registration system. Every patient arriving at the hospital receives a registration number. The hospital management wants a Java application that maintains patient records efficiently while preventing duplicate registrations.

Each patient record consists of

- ||Patient ID
- ||Patient Name
- ||Mobile Number
- ||Department
- ||Doctor Name

The system should utilize Java Collection Framework and String manipulation techniques for storing, validating, searching, and displaying patient information.

Functional Requirements

Develop a Java program using ArrayList and HashMap to perform the following operations.

Menu

1. Register Patient
2. Search Patient
3. Display All Patients
4. Count Department-wise Patients
5. Exit

Requirement 1 – Register Patient (4 Marks)

Accept

Patient ID

Patient Name

Mobile Number

Department

Doctor Name

Perform validations.

- ||Patient ID should be unique.
- ||Patient Name should contain only alphabets.
- ||Mobile Number should contain exactly 10 digits.
- ||Department should automatically convert to uppercase.
- ||Remove unnecessary spaces from all string inputs.

Store the patient in

- ||ArrayList
- ||HashMap (PatientID as Key)

Requirement 2 – Search Patient (4 Marks)

Accept Patient ID.

Display complete patient details.

If not found,

Patient Not Found

Requirement 3 – Display All Patients (4 Marks)

Display in the following format.

ID	Name	Mobile	Department	Doctor
P101	Rahul	9876543210	CARDIOLOGY	Dr.Arun
P102	Priya	9876543211	ORTHO	Dr.Meena

Requirement 4 – Department-wise Count (4 Marks)

Count the number of patients in each department.

Example

CARDIOLOGY : 5

ORTHO : 3

NEURO : 4

Use

HashMap<String,Integer>

Requirement 5 – String Operations (4 Marks)

For every patient,

Perform

- ||Convert Name to Proper Case
- ||Remove leading/trailing spaces
- ||Display Name Length
- ||Display Department in Uppercase

Example

Input

raHUI kumAr

Example
Input
raHUI kumAr
Output
Rahul Kumar
Length : 11
Department : CARDIOLOGY
Constraints
Maximum Patients = 100
Duplicate Patient IDs are not allowed.
Collections must be used.
Arrays are not allowed.
No Database.
Use only Java Collections.

Sample Input

1
P101
Rahul Kumar
9876543210
Cardiology
Dr.Arun

1
P102
Priya
9876543211
Ortho
Dr.Meena

3

4

2

P101

5

Sample Output

Patient Registered Successfully.

Patient Registered Successfully.

ID Name Department

P101 Rahul Kumar CARDIOLOGY
P102 Priya ORTHO

Department Count

CARDIOLOGY : 1

ORTHO : 1

Patient Details

ID : P101

Name : Rahul Kumar

Mobile : 9876543210

Department : CARDIOLOGY

Doctor : Dr.Arun

Your Code

```
1 import java.util.*;  
2  
3 public class Main {  
4  
5     static ArrayList<Patient> patients = new ArrayList<>();  
6     static HashMap<String, Patient> patientMap = new HashMap<>();  
7  
8     public static void main(String[] args) {  
9  
10         Scanner sc = new Scanner(System.in);  
11  
12         int n = Integer.parseInt(sc.nextLine());
```

Your Code

```

1 import java.util.*;
2
3 public class Main {
4
5     static ArrayList<Patient> patients = new ArrayList<>();
6     static HashMap<String, Patient> patientMap = new HashMap<>();
7
8     public static void main(String[] args) {
9
10         Scanner sc = new Scanner(System.in);
11
12         int n = Integer.parseInt(sc.nextLine());
13
14         for (int i = 0; i < n; i++) {
15             register(sc);
16         }
17
18         displayAll();
19     }
20
21
22     public static void register(Scanner sc) {
23
24         if (patients.size() >= 100) {
25             System.out.println("Maximum Patient Limit Reached");
26             return;
27         }
28
29         String id = sc.nextLine().trim();
30
31         if (patientMap.containsKey(id)) {
32             System.out.println("Duplicate Patient ID");
33             return;
34         }
35
36         String name = sc.nextLine().trim();
37         String mobile = sc.nextLine().trim();
38         String department = sc.nextLine().trim().toUpperCase();
39         String doctor = sc.nextLine().trim();
40
41         if (!name.matches("[A-Za-z ]+")) {
42             System.out.println("Invalid Patient Name");
43             return;
44         }
45
46         if (!mobile.matches("\\d{10}")) {
47             System.out.println("Invalid Mobile Number");
48             return;
49         }
50
51         Patient p = new Patient(id, name, mobile, department, doctor);
52
53         patients.add(p);
54         patientMap.put(id, p);
55
56         System.out.println("Patient Registered Successfully.");
57     }
58
59
60     public static void displayAll() {
61
62         for (Patient p : patients) {
63
64             System.out.println("ID : " + p.id);
65             System.out.println("Name : " + p.name);
66             System.out.println("Mobile : " + p.mobile);
67             System.out.println("Department : " + p.department);
68             System.out.println("Doctor : " + p.doctor);
69         }
70     }
71 }
72
73
74 class Patient {
75
76     String id;
77     String name;
78     String mobile;
79     String department;
80     String doctor;
81
82
83     Patient(String id, String name, String mobile, String department, String doctor) {
84
85         this.id = id;
86         this.name = properCase(name);
87         this.mobile = mobile;
88         this.department = department;
89         this.doctor = doctor;
90     }
91
92     private String properCase(String name) {
93         String[] words = name.split(" ");
94         for (int i = 0; i < words.length; i++) {
95             words[i] = words[i].charAt(0).toUpperCase() + words[i].substring(1);
96         }
97         return String.join(" ", words);
98     }
99 }

```

```

47         System.out.println("Invalid Mobile Number");
48         return;
49     }
50
51     Patient p = new Patient(id, name, mobile, department, doctor);
52
53     patients.add(p);
54     patientMap.put(id, p);
55
56     System.out.println("Patient Registered Successfully.");
57 }
58
59
60 public static void displayAll() {
61
62     for (Patient p : patients) {
63
64         System.out.println("ID : " + p.id);
65         System.out.println("Name : " + p.name);
66         System.out.println("Mobile : " + p.mobile);
67         System.out.println("Department : " + p.department);
68         System.out.println("Doctor : " + p.doctor);
69     }
70 }
71 }
72
73
74 class Patient {
75
76     String id;
77     String name;
78     String mobile;
79     String department;
80     String doctor;
81
82
83     Patient(String id, String name, String mobile, String department, String doctor) {
84
85         this.id = id;
86         this.name = properCase(name);
87         this.mobile = mobile;
88         this.department = department;
89         this.doctor = doctor;
90     }
91
92
93     static String properCase(String s) {
94
95         String[] words = s.toLowerCase().split("\\s+");
96
97         StringBuilder sb = new StringBuilder();
98
99         for (String w : words) {
100
101             sb.append(Character.toUpperCase(w.charAt(0)));
102             sb.append(w.substring(1));
103             sb.append(" ");
104         }
105
106         return sb.toString().trim();
107     }
108 }

```

Input

```

2
P101
John Smith
9876543210

```

Output

```

Patient Registered Successfully.
Patient Registered Successfully.
ID : P101
Name : John Smith
Mobile : 9876543210
Department : CARDIOLOGY

```

Visible Test Cases

Test Case 1	✖
Test Case 2	✖
Test Case 3	✖

0.00 / 20 marks

QUESTIONS

Q1

Smart Hospital Patient
Registration System
20 marks



Q2

Online Library Book
Management System
20 marks



2/2 answered

Q2 Online Library Book Management System

20 marks

Scenario

Online Library Book Management System

A university library is planning to automate its book management process. The librarian wants a Java application that stores book details, prevents duplicate book entries, allows students to search for books quickly, and generates category-wise reports.

The application should use Java String handling and Java Collection Framework to efficiently manage the library records.

Requirements

Each book contains the following details:

- Book ID
- Book Title
- Author Name
- Category
- Publisher

Use:

- ArrayList to store all book objects.
- HashMap to store Book ID as the key for quick searching.
- HashSet to maintain unique author names.

Menu Driven Program

1. Add Book
2. Search Book by ID
3. Display All Books
4. Display Unique Authors
5. Category-wise Book Count
6. Exit

Requirement 1 – Add Book (4 Marks)

Accept

- Book ID
- Book Title
- Author Name
- Category
- Publisher

Validate the following:

- Book ID must be unique.
- Remove leading and trailing spaces.
- Book Title should be stored in Title Case.
- Category should be converted to uppercase.
- Publisher name should not be empty.

Store the record in both

- ArrayList
- HashMap

Requirement 2 – Search Book (3 Marks)

Accept Book ID.

If found, display

- Book ID
- Book Title
- Author
- Category
- Publisher

Otherwise,

Book Not Available

Requirement 3 – Display All Books (4 Marks)

Display all records in tabular format.

Example

ID TITLE AUTHOR CATEGORY

B101 Java Programming Herbert Schildt PROGRAMMING
B102 Data Structures Mark Allen COMPUTER SCIENCE

Requirement 4 – Display Unique Authors (3 Marks)

Display all distinct author names using HashSet.

Example

Unique Authors

Herbert Schildt

James Gosling

Yashavant Kanetkar

Mark Allen

Requirement 5 – Category-wise Book Count (4 Marks)

Display the number of books available in each category.

Mark Allen

Requirement 5 – Category-wise Book Count (4 Marks)

Display the number of books available in each category using
HashMap<String,Integer>

Example

PROGRAMMING : 12

DATABASE : 8

NETWORKING : 6

Requirement 6 – String Operations (2 Marks)

For every book,

Display

☐Length of Book Title

☐Number of words in the title

☐Category in uppercase

Example

Input

Java Programming Language

Output

Length : 25

Words : 3

Category : PROGRAMMING

Sample Input

1

B101

Java Programming

Herbert Schildt

Programming

McGraw Hill

1

B102

Data Structures

Mark Allen

Computer Science

Pearson

3

4

5

2

B101

6

Sample Output

Book Added Successfully.

Book Added Successfully.

Book ID Title Author

B101 Java Programming Herbert Schildt

B102 Data Structures Mark Allen

Unique Authors

Herbert Schildt

Mark Allen

Category-wise Count

PROGRAMMING : 1

COMPUTER SCIENCE : 1

Book Details

Book ID : B101

Title : Java Programming

Author : Herbert Schildt

Category : PROGRAMMING

Publisher : McGraw Hill

Constraints

Constraints

Maximum books = 500

Duplicate Book IDs are not allowed.

Arrays are not allowed.

Use only Java Collections.

Book search should be performed using HashMap.

Use HashSet for maintaining unique authors.

Your Code

```
1 import java.util.*;
2
3 public class Main {
4
5     static ArrayList<Book> bookList = new ArrayList<>();
6     static HashMap<String, Book> bookMap = new HashMap<>();
7     static LinkedHashSet<String> authorSet = new LinkedHashSet<>();
8
9     public static String toTitleCase(String str) {
10         str = str.trim().toLowerCase();
11         String[] words = str.split("\\s+");
12         StringBuilder sb = new StringBuilder();
13
14         for (String w : words) {
15             sb.append(Character.toUpperCase(w.charAt(0)));
16             sb.append(w.substring(1));
17             sb.append(" ");
18         }
19
20         return sb.toString().trim();
21     }
22
23     public static void addBook(Scanner sc) {
24
25         String id = sc.nextLine().trim();
26
27         if (bookMap.containsKey(id)) {
28             System.out.println("Duplicate Book ID");
29             return;
30         }
31
32         String title = toTitleCase(sc.nextLine());
33
34         String author = sc.nextLine().trim();
35
36         String category = sc.nextLine().trim().toUpperCase();
37
38         String publisher = sc.nextLine().trim();
39
40         if (publisher.isEmpty()) {
41             System.out.println("Invalid Publisher");
42             return;
43         }
44
45         Book b = new Book(id, title, author, category, publisher);
46
47         bookList.add(b);
48         bookMap.put(id, b);
49         authorSet.add(author);
50
51         System.out.println("Book Added Successfully");
52     }
53
54     public static void searchBook(Scanner sc) {
55
56         String id = sc.nextLine().trim();
57
58         if (bookMap.containsKey(id)) {
59
60             Book b = bookMap.get(id);
61
62             System.out.println(b.getBookId());
63             System.out.println(b.getTitle());
64             System.out.println(b.getAuthor());
65             System.out.println(b.getCategory());
66             System.out.println(b.getPublisher());
67
68         } else {
69
70             System.out.println("Book Not Found");
71
72         }
73     }
74
75     public static void displayBooks() {
76
77         if (bookList.isEmpty()) {
78             System.out.println("No Books Available");
79             return;
80         }
81     }
82 }
```



```

79         return;
80     }
81
82     for (Book b : bookList) {
83         System.out.println(b.getBookId());
84         System.out.println(b.getTitle());
85         System.out.println(b.getAuthor());
86         System.out.println(b.getCategory());
87         System.out.println(b.getPublisher());
88     }
89 }
90
91 public static void displayAuthors() {
92
93     if (authorSet.isEmpty()) {
94         System.out.println("No Authors");
95         return;
96     }
97
98     for (String author : authorSet) {
99         System.out.println(author);
100     }
101 }
102
103 public static void categoryCount() {
104
105     LinkedHashMap<String, Integer> map = new LinkedHashMap<>();
106
107     for (Book b : bookList) {
108         String cat = b.getCategory();
109         map.put(cat, map.getOrDefault(cat, 0) + 1);
110     }
111
112     for (Map.Entry<String, Integer> e : map.entrySet()) {
113         System.out.println(e.getKey() + " : " + e.getValue());
114     }
115 }
116
117 public static void main(String[] args) {
118
119     Scanner sc = new Scanner(System.in);
120
121     while (true) {
122
123         int choice = Integer.parseInt(sc.nextLine());
124
125         switch (choice) {
126
127             case 1:
128                 addBook(sc);
129                 break;
130
131             case 2:
132                 searchBook(sc);
133                 break;
134
135             case 3:
136                 displayBooks();
137                 break;
138
139             case 4:
140                 displayAuthors();
141                 break;
142
143             case 5:
144                 categoryCount();
145                 break;
146
147             case 6:
148                 return;
149
150             default:
151                 System.out.println("Invalid Choice");
152         }
153     }
154 }
155 }
156
157 class Book {
158
159     private String bookId;
160     private String title;
161     private String author;
162     private String category;
163     private String publisher;
164
165     public Book(String bookId, String title, String author,
166                 String category, String publisher) {
167
168         this.bookId = bookId;
169         this.title = title;

```



```

149
150         default:
151             System.out.println("Invalid Choice");
152     }
153 }
154 }
155 }
156
157 class Book {
158
159     private String bookId;
160     private String title;
161     private String author;
162     private String category;
163     private String publisher;
164
165     public Book(String bookId, String title, String author,
166                 String category, String publisher) {
167
168         this.bookId = bookId;
169         this.title = title;
170         this.author = author;
171         this.category = category;
172         this.publisher = publisher;
173     }
174
175     public String getBookId() {
176         return bookId;
177     }
178
179     public String getTitle() {
180         return title;
181     }
182
183     public String getAuthor() {
184         return author;
185     }
186
187     public String getCategory() {
188         return category;
189     }
190
191     public String getPublisher() {
192         return publisher;
193     }
194
195     public void setBookId(String bookId) {
196         this.bookId = bookId;
197     }
198
199     public void setTitle(String title) {
200         this.title = title;
201     }
202
203     public void setAuthor(String author) {
204         this.author = author;
205     }
206
207     public void setCategory(String category) {
208         this.category = category;
209     }
210
211     public void setPublisher(String publisher) {
212         this.publisher = publisher;
213     }
214 }

```

Input

```

1
B101
Java Programming
Herbert Schildt

```

Output

```

Book Added Successfully
Book Added Successfully
B101
Java Programming
Herbert Schildt
PROGRAMMING

```

Visible Test Cases

Test Case 1	✖
Test Case 2	✖
Test Case 3	✖

0.00 / 20 marks